

Multi-Agent Job Application Pipeline – Design Document

1. System Overview

This system implements a multi-agent workflow designed to automate and optimize the job application process. The pipeline generates tailored offer packets, critiques outputs for quality, produces recruiter outreach messages, and tracks applications in a structured and scalable manner.

The architecture follows a controller-based orchestration model combined with a generate–critique–revise loop to ensure high-quality outputs.

2. Architecture & Design

The system consists of multiple specialized agents coordinated through a central orchestrator. Each agent performs a distinct role and communicates through a shared state.

Key architectural components include:

- Controller-based orchestration (Orchestrator Agent)
- Modular specialized agents (Offer, Critic, Outreach, Tracker)
- External tools/APIs for data enrichment
- Shared state for communication and memory

3. Agents and Roles

Orchestrator Agent (Controller)

Responsible for interpreting user input, classifying intent, and routing tasks to the appropriate agents. It ensures correct workflow execution.

Offer Packet Agent (Executor)

Generates customized job application materials (offer packets) based on job descriptions and user profile. Uses external APIs to enrich data.

Critic / Quality Agent

Evaluates generated outputs using scoring logic. Determines whether revisions are needed based on quality thresholds.

Outreach Agent

Creates recruiter outreach messages tailored to the job and company.

Tracker Agent

Logs applications, tracks progress, and stores structured data for analysis.

4. External Tools & APIs

SERP API – Used to research companies, roles, and contextual data.

Firecrawl – Extracts structured job descriptions from web pages.

5. Workflow & Data Flow

1. User input enters through chat trigger.
2. Orchestrator Agent classifies intent and routes task.
3. Offer Packet Agent generates initial output.
4. Critic Agent evaluates output and assigns score.
5. If $\text{score} < 7$ and $\text{revision_count} < 2 \rightarrow$ system loops back for revision.
6. If $\text{score} \geq 7 \rightarrow$ Outreach Agent generates recruiter message.
7. Tracker Agent logs application data.
8. Final Output is produced.

6. Agent Communication

Agents communicate through a shared state that includes:

- chat_history – stores user interaction context
- application_log – tracks all generated applications
- revision_count – tracks iteration cycles

This shared state ensures consistency, traceability, and coordination across agents.

7. Failure Handling & Quality Control

The system includes built-in safeguards to handle low-quality outputs:

- Outputs are scored by the Critic Agent.
- If score is below threshold, revision loop is triggered.
- Revision loop is capped (e.g., max 2 iterations).
- If still low quality after max attempts, best available output is returned.

This ensures efficiency while maintaining acceptable output quality.

8. Scalability Considerations

The modular design allows easy addition of new agents (e.g., Resume Agent, Interview Prep Agent).

External tools can be swapped or extended without affecting core workflow.

The system can be deployed as an API-based service for high-volume applications.

9. Output Structure

Final outputs include:

- Tailored Offer Packet
- Recruiter Outreach Message
- Application Tracking Record

Outputs are structured for easy readability and minimal cognitive load.

10. Conclusion

This multi-agent system provides a scalable, efficient, and intelligent solution for automating job applications. By combining orchestration, specialized agents, and iterative refinement, the system

ensures high-quality outputs and strong alignment with user goals.